

Problem Statement

The current checkout process offers limited payment options, resulting in a high rate of cart abandonment at the payment stage. Internal analytics indicate that a significant portion of drop-offs occur during payment selection, directly impacting conversion rates and overall revenue realization.

Additionally, the absence of an automated reconciliation mechanism between order data and payment transactions requires the Finance team to manually compile and validate transaction records. This process is time-consuming, error-prone, and delays financial reporting.

The lack of diverse, customer-preferred payment methods and automated reconciliation capabilities limits the organisation's competitiveness, reduces operational efficiency, and impacts financial visibility.

Business Goals

- Enhance checkout experience by expanding payment options.
- Improve revenue realization by optimizing payment-stage conversion.
- Strengthen financial governance through reconciliation automation.
- Improve operational efficiency within Finance operations.

KPIs

- Increase checkout conversion rate by 2.5% within two quarters post-implementation.
- Reduce payment-stage cart abandonment by at least 10% within six months.
- Achieve $\geq 95\%$ wallet transaction completion success rate.
- Reduce manual reconciliation effort by 65% within three months of go-live.
- Achieve 99.95% reconciliation accuracy for wallet transactions.

Business Requirements

- The organization shall introduce an internal wallet payment option within the checkout process to provide customers with an additional digital payment method.
- The wallet payment option shall enhance checkout flexibility and is expected to improve payment completion rates by reducing dependency on external payment gateways.
- The wallet system shall enable customers to store and reuse funds within the application to promote repeat transactions and long-term engagement.
- The wallet system shall ensure accurate, consistent, and tamper-proof recording of all wallet transactions, maintaining alignment between wallet balances and order payment statuses at all times.
- The system shall provide structured reporting and reconciliation capabilities for wallet transactions to support finance operations, auditing, and internal controls.

- The wallet functionality shall support structured handling of refunds, reversals, and transaction adjustments in accordance with the organization's refund policy.
- The wallet functionality shall comply with applicable country-level financial regulations, internal security policies, and data protection standards.
- Access to wallet functionality shall require an authenticated customer identity to ensure secure usage and protection of stored monetary value.

Stakeholder List

Primary Business Stakeholders

- Project Sponsor (e.g., Head of eCommerce / Digital)
- Product Owner
- CFO / Finance Head
- Legal & Compliance Officer
- Business Analyst / BA Lead

Operational Stakeholders

- Finance Team
- Operations Team
- Customer Support Team
- Risk & Fraud Team

Technical Stakeholders

- Development Team
- QA Team
- Solution Architect
- IT Security / Compliance Team
- DevOps / Infrastructure Team
- Data / BI Team

External Business Partners

- Wallet Provider (Technical + Account Manager)
- Payment Gateway / Acquiring Bank
- Third-party KYC / Identity Verification Provider

Governance Stakeholders

- Project Manager

In-scope

- Wallet as a Checkout Payment Option
 - Wallet displayed as a selectable payment method during checkout

- Available only for authenticated (registered & logged-in) users
- Wallet balance visibility during checkout
- Hide or disable wallet option if balance is zero or insufficient
- Minimum order value validation for wallet usage
- Wallet Transaction Handling
 - Debit wallet upon successful payment confirmation
 - Handle failed transactions gracefully with appropriate error messaging
 - Update order payment status accordingly (paid, failed, pending)
 - Retry mechanism for failed payments
 - Transaction timeout handling and status resolution
 - Duplicate transaction prevention / idempotency checks
 - Concurrent transaction handling to prevent race conditions
- Wallet Balance Management
 - Real-time balance validation before payment initiation
 - Accurate balance updates post-transaction
 - Prevention of negative balance
 - Concurrent transaction safeguards (locking mechanism)
 - Wallet transaction limits (daily spend limit, single transaction limit)
- Refund to Wallet
 - Refund eligible orders back to wallet
 - Partial refund to wallet support
 - Refund processing timeline adherence
 - Maintain refund transaction history
 - Align with business refund policy
 - Notify user upon successful refund credit
- Wallet Balance Management - Top-Up / Recharge
 - Wallet recharge via payment gateway (credit/debit card, net banking, UPI)
 - Minimum and maximum top-up limit enforcement
 - Top-up transaction history
- Notifications & Alerts
 - SMS / email / push notification for every debit transaction
 - Notification for successful refund credit
 - Notification for failed transaction with reason
 - Low balance alert
- Wallet Transaction Limits
 - Configurable daily wallet spend limit per user
 - Single transaction spend limit
 - Maximum wallet balance cap (if regulatory requirement applies)

- Reporting & Reconciliation
 - Wallet transaction reports (debit, credit, refund, failed)
 - Order-to-wallet transaction mapping
 - Finance reconciliation support (daily / end-of-day)
 - Exception report for unreconciled transactions
 - Role-based access control for reports (finance team, admin)
 - Reconciliation frequency definition (real-time / daily batch)
- Authentication Requirement
 - Wallet accessible only to registered and logged-in users
 - Role-based access - customers access own wallet only
 - Admin access to wallet transaction data for support and reconciliation
- Audit Trail & Logging
 - Maintain complete transaction logs (debit, credit, refund, failed attempts)
 - Track all wallet events with timestamp and user reference
 - Immutable / tamper-proof audit logs
 - Defined log retention period (as per compliance requirement)
 - Role-based access to audit logs (admin, finance, compliance team)
- Security & Compliance
 - Encrypted wallet balance and transaction data
 - PCI-DSS compliance for any payment data handled
 - KYC validation linkage (if regulatory requirement applies)
 - Fraud detection triggers for suspicious wallet activity
 - Session-based wallet access (no persistent token for wallet operations)

Out-of-scope

- Wallet Funding & External Transfers
 - Peer-to-peer (P2P) wallet transfers between users
 - Bank account withdrawal from wallet
 - Wire transfer or NEFT/RTGS from wallet
- Guest & Anonymous Users
 - Wallet usage during guest checkout
 - Wallet creation for non-registered users
- Multi-Currency & International
 - Multi-currency wallet support
 - Cross-border or international wallet transactions
 - Foreign exchange (FX) conversion within wallet

- Loyalty, Rewards & Promotions
 - Cashback crediting to wallet (*unless explicitly in scope*)
 - Loyalty points conversion to wallet balance
 - Promotional voucher or coupon integration with wallet
- Third-Party Wallet Integration
 - Integration with third-party wallets (Paytm, PhonePe, Google Pay, Apple Pay)
 - UPI-based wallet linking
 - Co-branded wallet programs
- Advanced Fraud & Risk Features
 - Real-time AI/ML-based fraud scoring
 - Velocity checks beyond basic transaction limits
 - Chargeback management workflow
- Advanced Reporting & Analytics
 - Business intelligence (BI) dashboards for wallet trends
 - Predictive analytics on wallet usage
 - Customer segmentation based on wallet behavior
- Customer-Facing Wallet Management
 - Wallet account closure / deactivation by user self-service
 - Wallet balance transfer to another account on closure
 - User-initiated wallet transaction dispute management portal

Assumptions

- **Business & Policy Assumptions**
 1. It is assumed that the business has finalized the wallet refund policy (eligible order types, refund window, and refund conditions) prior to development. If the refund policy is not finalized, refund-related requirements may be incomplete and could impact development timelines.
 2. It is assumed that the wallet will support user-initiated top-up (open-loop) as the primary funding mechanism, allowing users to load money via external payment methods (card, UPI, net banking). If the business decides to change to a closed-loop model, scope, architecture, and compliance requirements will need to be reassessed.
 3. It is assumed that wallet payment will only be available for full order value payments. Split or partial payments combining wallet and another payment method are excluded from this implementation.

4. It is assumed that the business will define and confirm wallet transaction limits (daily spend limit, single transaction limit, and maximum wallet balance cap) before development begins. If limits are not defined, default system limits will be applied as agreed with the business.
5. It is assumed that the refund processing timeline (instant or T+1) is governed by the existing business refund policy and will be followed as-is for wallet refunds. Any deviation from the existing policy for wallet-specific refunds will be treated as a new requirement.
6. It is assumed that the wallet payment option will be available across all product categories on the platform without any category-level restrictions.
7. It is assumed that the business will obtain or already holds the necessary regulatory approvals required to operate an open-loop wallet with user-initiated top-up. If regulatory approvals are pending, the top-up feature go-live may be subject to compliance clearance and could impact the overall project timeline.

- **Technical & Integration Assumptions**

1. It is assumed that the existing eCommerce platform supports wallet integration without requiring a major architectural overhaul. If significant platform changes are needed to accommodate wallet functionality, the effort will be assessed and treated as a separate technical workstream.
2. It is assumed that the existing payment gateway supports credit card transactions for wallet top-up. COD cannot be used as a wallet top-up method. If additional payment methods (UPI, net banking, debit card) are required for top-up, onboarding those methods will be treated as a separate scope item and may impact the project timeline.
3. It is assumed that the existing user authentication and session management system is stable and capable of supporting session-based wallet access controls. If the authentication system requires enhancements, it will be assessed separately.
4. It is assumed that the platform's existing database infrastructure can support real-time wallet balance updates and concurrent transaction handling. If additional database infrastructure is required (e.g., a distributed locking mechanism), it will be identified during technical design and flagged accordingly.
5. It is assumed that the existing order management system (OMS) exposes the necessary APIs or integration points to support wallet payment status updates and order-to-wallet transaction mapping. If OMS changes are required, they will be scoped and estimated separately.

6. It is assumed that the existing notification infrastructure (SMS, email, push) is available and will be extended to support wallet event-based notifications, including debit, credit, refund, and failed transaction alerts. No new notification infrastructure setup is required.
7. It is assumed that the platform has an existing logging and audit trail framework that can be extended to capture wallet-specific transaction events. If a new logging infrastructure is required, it will be assessed during technical design.
8. It is assumed that the wallet provider's API documentation, sandbox environment, and technical support are available at the start of the development phase. Any delays in API documentation or sandbox access from the wallet provider will be escalated as a project risk.
9. It is assumed that the existing platform supports idempotency mechanisms at the API layer to prevent duplicate wallet transactions. If this capability does not exist, it will need to be built as part of this implementation.
10. It is assumed that the platform's current infrastructure (servers, cloud environment) has sufficient capacity to handle wallet transaction volumes without performance degradation. If additional infrastructure is required, it will be identified during the non-functional requirements assessment.
11. It is assumed that full KYC verification is **not available** at the time of this implementation. The wallet will operate within regulatory transaction limits applicable to a minimum KYC wallet (as per RBI guidelines or applicable regulatory framework). If wallet usage exceeds defined thresholds, full KYC verification will be mandatory and will be treated as a separate workstream outside the scope of this project.
12. It is assumed that wallet refunds are primarily applicable for orders placed via **credit card payment**. For orders placed via **COD, the business has yet to confirm refund-to-wallet eligibility**. It is conditionally assumed that the business may support COD refunds to the wallet in the future. If COD-to-wallet refunds are confirmed during the project lifecycle, the refund eligibility rules, validation logic, and any additional processing requirements will be treated as an additional requirement and assessed for impact on scope and timeline.

- **Data Assumptions**

1. It is assumed that existing user account data is accurate, complete, and free of known quality issues based on current business knowledge. If data quality issues surface during implementation or testing, a data cleansing exercise will be initiated and treated as a separate workstream outside the project timeline.

2. It is assumed that each user will have **one wallet account** linked to their platform account. Multiple wallets per user account are not supported in this implementation. If multi-wallet support is required in the future, it will be treated as a separate enhancement.
3. It is assumed that all wallet transactions (debit, credit, refund, failed attempts, top-up) will be stored in a **dedicated wallet transaction table** separate from the existing order transaction records. This ensures clean data separation and simplifies reconciliation and audit trail requirements.
4. It is assumed that the existing order management system maintains accurate and up-to-date order status data that can be reliably referenced for wallet payment updates and refund eligibility validation. If order data integrity issues are identified, they will be flagged as a dependency risk.
5. It is assumed that the wallet will operate in a **single base currency** for this implementation. The business has not confirmed any multi-currency plans at this stage. If multi-currency support is identified as a future requirement, it will be treated as a separate enhancement and assessed for architectural and regulatory impact at that time.
6. The wallet transaction data retention period has **not yet been defined** by the business or compliance team. It is assumed that the retention period will be agreed upon and confirmed by the business, Finance, and Legal & Compliance stakeholders **prior to development commencing**. Until confirmed, a default industry-standard retention period of **5 years** will be used as a working assumption, in line with common financial data retention practices. Any deviation from this default must be formally documented and approved.
7. It is assumed that the Finance and Reconciliation team will have **read-only access** to wallet transaction data for reporting and reconciliation purposes. Write access to wallet transaction records will be restricted to system processes only to maintain data integrity and audit trail compliance.
8. It is assumed that wallet transaction records, once created, are **immutable** — meaning no manual edits or deletions are permitted on transaction records. Any corrections or reversals must be processed as a new transaction entry with a clear reference to the original transaction.
9. It is assumed that the existing reporting infrastructure can consume and process wallet transaction data for finance reconciliation reports. If new data pipelines or are required, they will be identified during technical design and assessed for effort.

10. It is assumed that a unique transaction reference ID will be generated for every wallet transaction to support traceability, reconciliation, and dispute resolution. The format and structure of the transaction reference ID will be defined and agreed upon with the technical team prior to development.
11. It is assumed that in the event of a failed transaction, the transaction record will still be captured in the system with a **failed status** and relevant failure reason code. Failed transaction records are important for audit, retry mechanisms, and reconciliation purposes and must not be discarded.
12. It is assumed that wallet top-up transaction data received from the payment gateway will include sufficient details (transaction ID, amount, timestamp, payment method, status) to support reconciliation between the payment gateway records and the internal wallet records.

- **Regulatory & Compliance Assumptions**

1. It is assumed that the business is currently **pursuing a Prepaid Payment Instrument (PPI) license from the RBI**. The wallet top-up feature will not go live until the license is formally granted. This is treated as a **critical dependency**, and any delays in licensing will directly impact the project's go-live timeline.
2. It is assumed that the wallet implementation will comply with **RBI guidelines on Prepaid Payment Instruments**, including transaction limits, user protection, and reporting obligations. Any compliance gaps identified during the project will be escalated to the Legal & Compliance Officer.
3. It is assumed that **KYC verification** is mandatory for users performing wallet top-up as per RBI requirements. Since KYC is not currently available, wallet top-up limits will be enforced within RBI's minimum KYC thresholds until full KYC is implemented. Full KYC will be treated as a separate workstream.
4. It is assumed that the wallet will comply with **RBI's data localisation requirements**, ensuring all payment transaction data is stored within India. If existing infrastructure is hosted outside India, data residency compliance will be treated as a critical dependency before go-live.
5. It is assumed that the business will implement basic **AML (Anti-Money Laundering)** controls in line with RBI and Financial Intelligence Unit India (FIU-IND) requirements. Advanced AML monitoring is out of scope and will be addressed as a separate compliance workstream.
6. It is assumed that **wallet transaction limits** (single transaction, daily, and cumulative) will be defined and enforced in line with RBI's prescribed limits for the applicable PPI category. Limits will be confirmed by the Legal & Compliance

Officer before development.

7. It is assumed that **GST compliance** for wallet transactions is governed by the existing tax framework on the platform. If additional GST configuration is required for wallet-specific transactions, it will be assessed separately.
8. It is assumed that **suspicious transaction reporting** to FIU-IND is an operational responsibility of the business compliance team. The system will provide the necessary transaction logs and exception reports to support this obligation.

- **User & Access Assumptions**

1. It is confirmed that the wallet feature will be available on **both web and mobile platforms** in line with the existing platform availability. No platform-specific restrictions apply.
2. It is assumed that each registered user will have **one wallet account** linked to their platform account. Multiple wallet accounts per user are not supported in this implementation.
3. It is assumed that wallet access is **session-based**. Upon session expiry or logout, wallet access is immediately revoked. Users must re-authenticate before accessing the wallet in a new session.
4. It is assumed that users can only view and operate their **own wallet**. Cross-account wallet access is not permitted under any circumstances.
5. It is confirmed that a **Role and Access Management system is already in place**. Wallet-specific roles and permissions will be configured within the existing system. No new access management infrastructure is required for this implementation.
6. It is assumed that the **Finance and Reconciliation team** will have read-only access to wallet transaction data for reporting and reconciliation purposes. They will not have write or modification access to any wallet transaction records.
7. It is assumed that the **Risk & Fraud team** will have access to wallet transaction logs and exception reports to monitor suspicious activity. Their access will be read-only and governed by role-based access controls.
8. It is assumed that the Customer Support team's access to **wallet transaction history** will follow the same access model as their existing order transaction access. No new access framework is required for wallet-specific data – the existing access model will be extended to include wallet transactions.

9. It is assumed that all **admin and internal user access** to wallet data will be logged in the audit trail for accountability and compliance purposes.
10. It is assumed that wallet features will be accessible via both **web and mobile platforms**. If wallet functionality needs to be restricted to a specific platform (web only or mobile only), it will be treated as an additional requirement.
11. It is assumed that a **user can voluntarily deactivate their wallet** only through a formal request process managed by Customer Support. Self-service wallet deactivation is out of scope for this implementation.

- **Third Party & Vendor Assumptions**

1. It is assumed that the **wallet technology provider selection is currently in progress**. Vendor identification and onboarding must be completed before development can commence. Until the vendor is confirmed, API design, integration planning, and sandbox testing cannot be initiated. This is treated as a **critical project dependency**, and any delays in vendor selection will directly impact the project timeline.
2. It is assumed that once the wallet provider is selected, they will supply **complete API documentation, a sandbox environment, and technical support** at the start of the development phase. Any delays in providing these post-selection will be escalated as a project risk.
3. It is assumed that wallet top-up will initially support credit card payments only, using the existing payment gateway. UPI, debit card, and net banking support for top-up will be assessed for inclusion in a future phase. If additional payment methods are required in the current phase, payment gateway expansion will be treated as a separate scope item.
4. It is assumed that the payment gateway provides **real-time transaction status notifications** (success, failure, timeout) via webhooks or callbacks to support accurate wallet balance updates and order status management.
5. It is assumed that the payment gateway supplies a **unique transaction reference ID** for every top-up transaction to support reconciliation between gateway records and internal wallet records.
6. It is assumed that **SLA discussions with the wallet provider** will be initiated once the vendor is confirmed and finalized before contract signing. SLA coverage must include system availability, transaction response time, and support response time. Unresolved SLAs at the time of contract signing will be escalated to the Project Sponsor.

7. It is assumed that the **notification service provider** (SMS, email, push) is already contracted and integrated with the platform. Wallet event-based notifications will be configured within the existing notification service without requiring a new vendor onboarding.
8. It is assumed that all third-party vendors involved in the wallet implementation are **compliant with RBI regulations** and applicable data protection requirements. Vendor compliance certificates will be obtained and reviewed by the Legal & Compliance Officer before go-live.
9. It is assumed that the wallet provider supports **sandbox testing** for all wallet operations, including top-up, debit, refund, and failed transaction scenarios, before production deployment. If sandbox coverage is incomplete, it will be flagged as a testing risk.
10. It is assumed that any **third-party vendor changes** (API version upgrades, pricing changes, service discontinuation) during the project lifecycle will be communicated with sufficient notice to allow the technical team to assess and accommodate the impact without disrupting the project timeline.
11. It is assumed that the wallet provider's **system availability SLA** will be assessed and confirmed against the platform's uptime requirements once the vendor is selected. If a gap is identified, mitigation measures such as fallback payment options will be discussed with the business before go-live.

- **Project & Timeline Assumptions**

1. It is assumed that **business stakeholders will be available** for requirements review, clarification, and sign-off within agreed timeframes. Delays in stakeholder availability will directly impact the project timeline.
2. It is assumed that the **project scope is stable** at the time of BRD sign-off. Any scope changes post-sign-off will be managed through a formal change control process and assessed for impact on timeline, cost, and resources.
3. It is assumed that **dedicated development, QA, and DevOps resources** will be allocated to the wallet project for the full duration of the implementation. Any resource changes or reallocation during the project will be escalated to the Project Sponsor.
4. It is assumed that the **wallet provider is selected and onboarded** within a defined timeframe to avoid impacting the development start date. Delays in vendor selection beyond the agreed date will be escalated as a critical project risk.

5. It is assumed that the **PPI license from RBI** will be granted before the wallet top-up feature go-live date. If the license is delayed, the top-up feature will be deferred, and a phased go-live will be planned accordingly.
6. It is assumed that a **UAT (User Acceptance Testing) environment** will be available and configured before the UAT phase. Any delays in environment setup will be treated as a dependency risk and escalated to the DevOps team.
7. It is assumed that **business stakeholders will actively participate in UAT** and provide sign-off within the agreed UAT window. Extended UAT cycles or delayed sign-off will impact the go-live date.
8. It is assumed that all **third-party API integrations** (wallet provider, payment gateway) will be available and stable in the test environment before the start of integration testing. Any instability or unavailability of third-party sandboxes will be flagged as a testing risk.
9. It is assumed that a **phased go-live approach** will be considered if not all features are ready simultaneously. For example, core wallet payment and refund features may go live before the top-up feature, subject to PPI license availability.
10. It is assumed that a **post-go-live hypercare period** will be agreed upon with the business and technical teams to monitor wallet transactions, resolve critical issues, and ensure system stability before handing over to BAU (Business as Usual) operations.
11. It is assumed that all **project dependencies** (PPI license, vendor selection, payment gateway expansion decision) that are currently open will be resolved within the agreed deadlines. Unresolved dependencies beyond their deadline will be escalated to the Project Sponsor for a decision.
12. It is assumed that a **formal BRD sign-off** will be obtained from all key stakeholders before development commences. Development starting without formal sign-off will be at the business's own risk, and any resulting rework will not be treated as a project defect.

Dependencies

- **Internal System Dependencies**

1. **User Authentication & Session Management System:** The wallet feature depends on the existing authentication system to validate user identity and manage session-based wallet access. Wallet operations cannot be performed outside an active, authenticated session.

Internal system dependencies are based on current knowledge. Additional

dependencies may be identified during the technical design phase and will be documented accordingly.

2. **Order Management System (OMS):** The wallet depends on the OMS to retrieve accurate order details for payment processing, refund eligibility validation, and order payment status updates. Any instability or API changes in the OMS will directly impact wallet transaction processing.
3. **Payment Gateway Integration:** The wallet top-up feature depends on the existing payment gateway for processing credit card transactions. The gateway must support wallet-specific transaction types and provide real-time transaction status callbacks.
4. **Notification System:** Wallet event-based notifications (debit, credit, refund, failed transaction alerts) depend on the existing notification infrastructure. The notification system must be capable of being extended to support wallet-specific events.
5. **Reporting & Reconciliation System:** Wallet transaction reports and finance reconciliation depend on the existing reporting infrastructure being able to consume and process wallet transaction data. Any limitations in the reporting system will impact reconciliation capability.
6. **Role & Access Management System:** Wallet-specific roles and permissions depend on the existing role and access management system. Wallet access controls will be configured within the existing framework without requiring a new system.
7. **Audit & Logging Framework:** The wallet audit trail depends on the existing logging framework being extended to capture wallet-specific transaction events. If the existing framework cannot support wallet logging requirements, a new solution will need to be identified.
8. **Fraud Detection Rules Engine:** Since no dedicated fraud detection or risk management system is currently available, basic rule-based fraud detection triggers for wallet transactions will be built as part of this implementation. The Risk & Fraud team must define and confirm fraud rules before development to avoid rework.

- **External / Third Party Dependencies**

1. **Wallet Technology Provider:** The entire wallet implementation depends on the selection and onboarding of a wallet technology provider. API documentation, sandbox environment, and technical support from the provider must be available before development can commence. This is the **highest priority external dependency** for the project.
2. **Payment Gateway – Credit Card Top-Up:** The wallet top-up feature depends on the existing payment gateway supporting credit card transactions for wallet funding. The gateway must provide real-time transaction status notifications (success, failure, timeout) via webhooks or callbacks.
3. **Payment Gateway – UPI & Debit Card Support.** If the business confirms UPI and debit card as additional top-up methods, the payment gateway must be expanded to support these payment methods. This expansion will be treated as a separate dependency and assessed for timeline impact.
4. **Wallet Provider Sandbox Environment:** Integration testing and QA activities depend on the wallet provider making a stable and fully functional sandbox environment available prior to the start of the development phase. Sandbox instability or limited coverage will directly impact testing quality and timeline.
5. **Payment Gateway Sandbox Environment:** Testing of wallet top-up transactions depends on the payment gateway sandbox environment being available and stable for the full duration of the testing phase. Any sandbox downtime or limitations must be communicated in advance.
6. **Wallet Provider SLA & Availability:** The wallet feature's production reliability depends on the wallet provider meeting agreed SLA commitments for system availability, transaction response time, and support response time. SLA terms must be finalized and agreed upon before contract signing.
7. **Third Party KYC Provider:** If the business decides to implement KYC verification as part of or immediately following this project, a third-party KYC provider will need to be identified, onboarded, and integrated. Until KYC is available, wallet top-up transactions will be restricted to RBI minimum KYC thresholds.
8. **SMS / Email / Push Notification Service Provider:** Wallet event-based notifications depend on the existing notification service provider being available and capable of supporting wallet-specific notification templates and event triggers. Any provider limitations must be identified and addressed before go-live.

9. **RBI Payment Aggregator Compliance:** If the existing payment gateway operates as a payment aggregator, it must be RBI-registered and compliant with RBI payment aggregator guidelines. Confirmation of the gateway's regulatory status is a dependency before finalizing the top-up architecture.
- **Regulatory & Compliance Dependencies**
 1. **PPI License from RBI:** The wallet top-up feature cannot go live without a valid **Prepaid Payment Instrument (PPI) license from RBI**. This is the single most critical regulatory dependency for the project. The business must secure this license before the top-up feature can be launched in production.
 2. **RBI Minimum KYC Compliance:** Until full KYC is implemented, the wallet must operate within **RBI's minimum KYC transaction thresholds** — currently limited to a maximum wallet balance of ₹10,000 and cumulative annual transactions of ₹1,00,000. The business and compliance team must confirm these limits are acceptable for the initial launch.
 3. **RBI Data Localisation Compliance:** The wallet depends on all payment transaction data being stored within India in compliance with **RBI data localisation guidelines**. It is confirmed that the platform infrastructure is hosted within India, satisfying this dependency.
 4. **GST Compliance Confirmation:** The wallet implementation depends on the Finance and Legal team confirming that **GST treatment for wallet transactions** (top-up, debit, refund) is clearly defined and handled correctly within the existing tax framework. Any gaps in GST handling must be resolved before go-live.
 5. **Legal & Compliance Sign-Off:** The wallet feature cannot go live without a formal **Legal & Compliance sign-off** confirming that all regulatory obligations have been met. This sign-off is a mandatory gate before production deployment.
 - **Data Dependencies**
 1. **User Account Data Availability:** The wallet implementation depends on accurate and complete user account data (user ID, name, email, phone number) being available in the existing system for wallet account creation and linkage. Incomplete or inaccurate user data will directly impact wallet onboarding and transaction processing.
 2. **Order Data Availability & Integrity:** Wallet refund processing and payment status updates depend on accurate and real-time order data being available from the Order Management System. Any gaps or inconsistencies in order data will impact refund eligibility validation and wallet balance updates.

3. **Transaction Reference ID Generation:** All wallet transactions depend on a **unique transaction reference ID** being generated at the point of transaction initiation. This ID is critical for reconciliation, audit trail, dispute resolution, and duplicate transaction prevention. The ID generation mechanism must be defined and agreed upon with the technical team before development.
4. **Payment Gateway Transaction Data:** Wallet top-up reconciliation depends on the payment gateway providing complete transaction data, including transaction ID, amount, timestamp, payment method, and status for every top-up transaction. Incomplete gateway data will create reconciliation gaps.
5. **Wallet Transaction Data Structure:** The reporting, reconciliation, and audit trail features depend on a well-defined **wallet transaction data structure** being agreed upon before development. This includes transaction type, status codes, failure reason codes, timestamps, and user references. Any changes to the data structure post-development will require rework.
6. **Failure Reason Code Definitions:** Failed transaction handling and reporting depend on a **standardized set of failure reason codes** being defined and agreed upon by the business and technical teams before development. These codes are essential for meaningful exception reports and customer-facing error messaging.
7. **Refund Eligibility Data:** Wallet refund processing depends on clear and accessible data defining **refund eligibility rules** – including eligible order types, refund window, and refund conditions — as governed by the existing business refund policy. If refund eligibility data is not clearly defined, automated refund processing cannot be built accurately.
8. **Wallet Balance Data Integrity:** All wallet operations depend on the wallet balance being **accurate and consistent at all times**. The system must ensure data integrity during concurrent transactions, system failures, and retry scenarios. Any balance discrepancy must be immediately flagged for reconciliation and resolution.
9. **Audit Log Data Retention:** The audit trail depends on a confirmed **data retention period** being defined by the business and compliance team before development. As agreed earlier, a default of 5 years will apply until the retention period is formally confirmed.
10. **Fraud Rule Data:** The rule-based fraud detection system depends on the **Risk & Fraud team defining and confirming fraud trigger rules** before development. Without confirmed fraud rules, the fraud detection capability cannot be built or tested accurately.

11. **Notification Template Data:** Wallet event-based notifications depend on **notification templates** (SMS, email, push) being defined and approved by the business before development. Templates must cover all wallet events, including debit, credit, refund, top-up, and failed transaction alerts.
12. **Reconciliation Data Mapping:** Finance reconciliation depends on a clear and agreed **data mapping** between wallet transaction records and payment gateway records. This mapping must be defined before development to ensure reconciliation reports are accurate and complete from day one.

- **People & Process Dependencies**

1. **Stakeholder Availability for Requirements Sign-Off:** The project depends on key business stakeholders being available to review, validate, and sign off on requirements within agreed timeframes. Delays in stakeholder availability will directly impact the project timeline and development start date.
2. **Business Decision on Top-Up Payment Methods:** The wallet top-up scope depends on the business confirming whether **UPI and debit cards** will be supported in addition to credit cards for the initial launch. This decision must be made before technical design to avoid rework on the payment gateway architecture.
3. **Business Decision on COD Refund to Wallet:** The refund scope depends on the business confirming whether **COD orders are eligible for wallet refunds**. This decision must be made before refund requirements are finalized and development commences.
4. **Fraud Rule Definition by Risk & Fraud Team:** The fraud detection capability depends on the **Risk & Fraud team defining and approving fraud trigger rules** before development. Without confirmed rules, fraud detection cannot be built or tested. This is a people dependency as it requires active involvement and decision-making from the Risk & Fraud team.
5. **Refund Policy Confirmation by Business:** The wallet refund feature depends on the business formally confirming the refund policy – including eligible order types, refund window, refund conditions, and processing timeline – before development. Any ambiguity in the refund policy will result in incorrect refund logic being built.
6. **Wallet Transaction Limits Confirmation:** The wallet balance management and compliance features depend on the **Legal & Compliance Officer and business confirming wallet transaction limits** – daily spend limit, single transaction limit, and maximum wallet balance cap – in line with RBI PPI guidelines before development.

7. **KYC Process Decision:** The wallet's regulatory compliance depends on the business and Legal & Compliance teams making a formal decision on the **KYC implementation approach** – whether to build a basic KYC process within this project or defer it to a separate workstream. This decision must be made early to avoid scope ambiguity.
 8. **UAT Participation by Business Stakeholders:** The go-live timeline depends on business stakeholders actively participating in **User Acceptance Testing (UAT)** and providing timely sign-off within the agreed UAT window. Delayed UAT participation or extended feedback cycles will directly impact the go-live date.
 9. **Customer Support Team Readiness:** The wallet go-live depends on the **Customer Support team being trained and operationally ready** to handle wallet-related customer queries, disputes, and escalations before the feature is launched. If the support team is not ready at go-live, customer experience will be negatively impacted.
 10. **Finance Team Process Readiness:** The wallet reconciliation feature depends on the **Finance team having defined and documented reconciliation processes** for wallet transactions before go-live. The system can generate reconciliation reports, but the Finance team must be operationally ready to act on them.
 11. **Vendor Contract Sign-Off:** The wallet provider onboarding depends on the **legal and procurement teams completing contract negotiations and sign-off** within the agreed timeline. Delays in contract finalization will directly delay the development start date.
 12. **Post Go-Live Hypercare Team Availability:** The stability of the wallet feature post launch depends on a **dedicated hypercare team** – including development, QA, and business representatives — being available to monitor, respond to, and resolve issues during the agreed hypercare period.
- **Infrastructure Dependencies**
 1. **Development Environment Readiness:** The wallet development depends on a **dedicated and fully configured development environment** being available at the start of the project. The environment must support wallet-specific configurations, third-party API connections, and database schema changes without impacting the existing production system.
 2. **Test / QA Environment Readiness:** Wallet testing depends on a **stable and isolated QA environment** that mirrors the production setup. The QA environment must support end-to-end wallet transaction testing, including top-up, debit, refund, and failed transaction scenarios.

3. **UAT Environment Readiness:** Business stakeholders depend on a **dedicated UAT environment** being configured and available before the UAT phase. The UAT environment must be populated with realistic test data and connected to third-party sandboxes for wallet provider and payment gateway testing.
4. **Production Environment Readiness:** The wallet go-live depends on the **production environment being configured, tested, and approved** by the DevOps and IT Security teams before deployment. Any gaps identified in the production environment during deployment rehearsal must be resolved before go-live.
5. **Database Infrastructure** The wallet implementation depends on the existing database infrastructure supporting:
 - a. Real-time wallet balance updates
 - b. Concurrent transaction handling
 - c. Distributed locking mechanisms to prevent race conditions
 - d. Dedicated wallet transaction tables are separate from existing order data

If the existing database infrastructure cannot support these requirements, upgrades will be treated as a critical infrastructure dependency.

6. **Server Capacity:** The wallet feature depends on the existing server infrastructure having **sufficient capacity** to handle wallet transaction volumes without performance degradation. A capacity assessment must be conducted before go-live to confirm infrastructure readiness.
7. **Data Hosting Within India:** As confirmed earlier, the wallet implementation depends on **all transaction and user data being hosted within India** in compliance with RBI data localisation requirements. Any infrastructure hosted outside India must be identified and remediated before go-live.
8. **API Gateway & Load Balancing:** The wallet's performance and reliability depend on the existing **API gateway and load balancing infrastructure** being capable of handling wallet API traffic volumes. If additional capacity or configuration is required, it must be addressed during the infrastructure readiness phase.
9. **Backup & Disaster Recovery:** The wallet implementation depends on a **backup and disaster recovery plan** being in place for wallet transaction data. Given the financial nature of wallet data, recovery point objectives (RPO) and recovery time objectives (RTO) must be defined and agreed upon with the business and DevOps team before go-live.

10. **SSL / TLS Encryption Infrastructure:** All wallet transactions depend on **SSL/TLS encryption being in place** for all data in transit between the platform, wallet provider, and payment gateway. If the existing encryption infrastructure does not cover wallet-specific endpoints, it must be extended before go-live.
11. **Monitoring & Alerting Infrastructure:** The wallet's operational stability post go-live depends on a **monitoring and alerting system** being configured to track wallet transaction success rates, system errors, response times, and balance discrepancies in real time. Any gaps in monitoring coverage must be addressed before go-live.
12. **CI/CD Pipeline Readiness:** The wallet deployment depends on the existing **CI/CD (Continuous Integration / Continuous Deployment) pipeline** being configured to support wallet-specific build, test, and deployment workflows. If the pipeline requires additional configuration for wallet components, it must be completed before the first deployment cycle.

Constraints:

- **Regulatory & Legal Constraints**

1. **RBI PPI License Compliance:** The wallet must operate strictly within the boundaries of RBI's Prepaid Payment Instrument (PPI) regulations. Any wallet feature that conflicts with RBI PPI guidelines cannot be implemented, regardless of business preference.
2. **RBI Minimum KYC Transaction Limits:** Until full KYC is implemented, the wallet must enforce RBI's minimum KYC transaction thresholds — maximum wallet balance of ₹10,000 and cumulative annual transactions of ₹1,00,000. These limits are non-negotiable and cannot be overridden by the business.
3. **RBI Data Localisation:** All wallet transactions and user data must be stored within India in compliance with RBI data localisation requirements. Data cannot be hosted or processed outside India under any circumstances.
4. **AML & CFT Compliance:** The wallet must comply with AML and CFT regulations as governed by RBI and FIU-IND. Suspicious transaction monitoring and reporting obligations are mandatory and cannot be deferred or excluded from the implementation.
5. **GST Compliance:** All wallet transactions must comply with Indian GST regulations as governed by the Federal Tax Authority. Wallet features cannot go live without confirmed GST treatment for all transaction types.

6. **Data Retention Compliance:** Wallet transaction data must be retained for a minimum of 5 years in line with RBI financial data retention guidelines. Data cannot be deleted or purged before the minimum retention period expires.

- **Technical Constraints**

1. **Existing Platform Architecture:** The wallet must be built within the boundaries of the existing platform architecture. Any changes that require a major architectural overhaul are out of scope for this implementation and will be treated as a separate technical initiative.
2. **Existing Payment Gateway:** The wallet top-up feature is constrained to the capabilities of the existing payment gateway. Payment methods not currently supported by the gateway cannot be offered for wallet top-up without a separate gateway expansion initiative.
3. **Existing Authentication System:** Wallet access controls must be built within the boundaries of the existing authentication and session management system. A new authentication framework cannot be introduced as part of this implementation.
4. **Existing Role & Access Management System:** Wallet-specific roles and permissions must be configured within the existing role and access management system. A new access management framework cannot be introduced for wallet-specific requirements.
5. **Single Wallet Per User:** The system architecture will support only one wallet account per registered user. Multi-wallet functionality is a technical constraint of this implementation and cannot be accommodated without significant architectural changes.
6. **Single Base Currency:** The wallet will operate in a single base currency (INR) only. Multi-currency support is a technical constraint of this implementation and will require a separate architectural assessment if needed in the future.

- **Business Constraints**

1. **Fixed Budget:** The wallet implementation must be delivered within the approved fixed budget. Any scope additions or changes that require budget beyond the approved amount must go through formal change control and receive budget approval before proceeding.
2. **Scope Freeze Post Sign-Off:** Once the BRD is formally signed off, the project scope is frozen. Any new requirements or scope changes introduced after sign-off will be subject to formal change control assessment for budget, timeline,

and resource impact.

3. **Phased Go-Live:** If the PPI license is not secured before the planned go-live date, the wallet will launch in phases — core wallet payment and refund features first, followed by top-up once regulatory approval is granted. The business cannot launch a top-up without a valid PPI license, regardless of project readiness.
 4. **Existing Refund Policy:** The wallet refund feature must align with the existing business refund policy. The wallet implementation cannot introduce new refund rules or timelines that conflict with the existing policy without formal business approval.
 5. **Wallet Transaction Limits:** Wallet transaction limits (daily spend, single transaction, maximum balance) must align with RBI PPI guidelines and cannot be set above regulatory thresholds, regardless of business preference
- **Security Constraints**
 1. **Session-Based Wallet Access:** Wallet access must be strictly session-based. Persistent tokens or long-lived authentication mechanisms cannot be used for wallet operations under any circumstance.
 2. **Data Encryption:** All wallet transaction data must be encrypted both in transit (SSL/TLS) and at rest. Unencrypted storage or transmission of wallet data is not permitted under any circumstances.
 3. **Immutable Transaction Records:** Wallet transaction records cannot be manually edited, modified, or deleted once created. Any corrections must be processed as new transactions with a clear reference to the original record.
 4. **Role-Based Access Control:** Access to wallet data must be strictly role-based. Unauthorized access to wallet transaction data by any internal or external party is not permitted under any circumstances.
 5. **PCI-DSS Compliance:** Any payment card data handled during wallet top-up must comply with PCI-DSS standards. Non-compliant handling of card data is not permitted, regardless of implementation convenience.
 6. **Fraud Detection Controls:** Basic rule-based fraud detection controls are mandatory for the wallet implementation. The wallet cannot go live without at least a defined set of fraud trigger rules agreed upon by the Risk & Fraud team.
 - **Data Constraints**

1. **Data Separation:** Wallet transaction data must be stored in a dedicated wallet transaction table separate from existing order transaction records. Commingling of wallet and order data is not permitted as it compromises reconciliation integrity.
2. **Data Immutability:** Once a wallet transaction record is created, it cannot be altered or deleted. This is a non-negotiable data integrity and audit compliance constraint.
3. **Minimum Data Retention Period:** Wallet transaction data must be retained for a minimum of 5 years. Early deletion or purging of wallet data before the retention period expires is not permitted under any circumstances.
4. **Data Access Restriction:** Write access to wallet transaction records is restricted to system processes only. No internal user – including Finance, Customer Support, or Admin – can directly modify wallet transaction data.
5. **Single Currency Data Storage:** All wallet balance and transaction amounts must be stored in INR only. Multi-currency data storage is not supported in this implementation.

Non-functional requirements:

1. Performance:

- a. **Wallet Payment Response Time:** The wallet debit transaction must be completed and a response returned to the user within **3 seconds** under normal load conditions. This includes balance validation, transaction processing, and order status update.
- b. **Wallet Top-Up Response Time:** The wallet top-up transaction must be completed and wallet balance updated within **5 seconds** of payment gateway confirmation under normal load conditions.
- c. **Wallet Balance Display Response Time:** The wallet balance must be displayed to the user during checkout within **1 second** of page load under normal load conditions.
- d. **Refund Processing Time:** Wallet refunds must be credited to the user's wallet within **5 minutes** of refund approval by the system. This excludes manual refund approvals which are subject to business process timelines.
- e. **Concurrent Transaction Handling:** The system must support a minimum of **500 concurrent wallet transactions** without performance degradation during peak load conditions.
- f. **Transaction Throughput:** The wallet system must support a minimum throughput of **100 TPS under normal load** and **250 TPS during peak load** conditions such as sales events and promotions
- g. **Notification Delivery Time:** Wallet event-based notifications (SMS, email, push) must be delivered to the user within **30 seconds** of the triggering wallet event

under normal conditions.

2. Scalability:

- a. The wallet system must be designed to accommodate increased transaction volumes without service interruption or performance degradation. Scaling must not require system downtime or impact the end user experience.
- b. The wallet system must sustain a minimum of 3x growth in transaction volume from the initial launch baseline while maintaining defined performance and response time benchmarks.
- c. The wallet system must maintain consistent performance and response times for up to 1 million registered wallet users without service degradation.
- d. The wallet system must maintain consistent query and reporting performance as historical transaction data accumulates over time. Performance benchmarks defined in this document must be sustained throughout the data retention period.

3. Availability & Reliability:

- a. **System Availability:** The wallet system must maintain a minimum uptime of **99.9%** — equivalent to a maximum of approximately 8.7 hours of unplanned downtime per year. This applies to all wallet operations including payment, top-up, refund, and balance display.
- b. **Planned Maintenance Window:** Planned maintenance and system upgrades must be scheduled during **off-peak hours** (e.g. 2:00 AM - 5:00 AM IST) and communicated to users at least 24 hours in advance. Wallet operations must be gracefully suspended during maintenance with appropriate user messaging.
- c. **Transaction Failure Rate:** The wallet system must maintain a transaction failure rate of **no more than 0.1%** under normal operating conditions. Failure rates exceeding this threshold must trigger an automated alert to the operations and technical teams.
- d. **Automatic Failover** In the event of a system failure, wallet services must be automatically restored within **30 seconds** to minimize transaction disruption and avoid user-facing errors.
- e. **Transaction Atomicity** Every wallet transaction must either complete fully or be reversed entirely. Partial transaction states that result in balance discrepancies or incomplete order updates are not permitted under any circumstance.
- f. **Retry Mechanism Reliability** The automated retry mechanism for failed transactions must successfully recover and complete a minimum of **80% of retryable failures** without manual intervention.

4. Security:

- a. **Data Encryption in Transit:** All wallet transaction data transmitted between the platform, wallet provider, and payment gateway must be encrypted using **TLS 1.2 or higher**. Unencrypted data transmission is not permitted under any circumstance.

- b. **Data Encryption at Rest:** All wallet transaction data and user wallet information stored in the database must be encrypted at rest using **AES-256 encryption** as a minimum standard.
- c. **PCI-DSS Compliance:** All payment card data handled during wallet top-up must comply with **PCI-DSS Level 1** standards — the highest level of payment card data security compliance.
- d. **Session Timeout:** Wallet sessions must automatically expire after **15 minutes of inactivity**. Users must re-authenticate before resuming wallet operations after session expiry.
- e. **Failed Authentication Lockout:** After **5 consecutive failed authentication attempts**, the user account must be temporarily locked for a minimum of **30 minutes**. The user must be notified via SMS or email upon account lockout.
- f. **API Security:** All wallet APIs must be secured through industry-standard authentication mechanisms. API credentials must be rotated at regular intervals of no more than **90 days** to minimize the risk of unauthorized access. All unauthorized API access attempts must be logged and flagged immediately for security review.
- g. **Vulnerability Assessment:** A penetration test and security vulnerability assessment must be conducted on the wallet system prior to go-live. All critical and high severity vulnerabilities must be resolved before production deployment.
- h. **Fraud Detection Response Time:** Rule-based fraud detection triggers must evaluate and respond to suspicious wallet transactions within **2 seconds** of transaction initiation to prevent fraudulent transactions from completing.

5. Usability:

- a. **Checkout Integration Visibility:** The wallet payment option must be clearly visible and easily accessible within the payment selection section during checkout on both web and mobile platforms without requiring excessive scrolling or navigation.
- b. **Error Messaging:** All wallet transaction failures must display a **clear, user-friendly error message** explaining the reason for failure and suggested next steps. Technical error codes must not be displayed directly to end users.
- c. **Mobile Responsiveness:** The wallet feature must be fully responsive and functional across all screen sizes on both **iOS and Android** mobile platforms. Wallet UI elements must adapt to screen size without layout breaks or functional limitations.
- d. **Transaction Confirmation:** Users must receive an **on-screen confirmation message** within 3 seconds of every successful wallet transaction — including payment, top-up, and refund — clearly showing the transaction amount, updated wallet balance, and transaction reference ID.

6. Maintainability:

- a. **Code Maintainability:** The wallet system must be built to established coding and documentation standards to ensure the codebase is maintainable, extensible, and supportable by the internal technology team beyond the initial delivery.
- b. **Configuration Management:** Wallet transaction limits, fraud rules, and notification templates must be **configurable via an admin interface** without requiring code changes or system redeployment. This allows the business to adjust parameters quickly in response to regulatory or operational changes.
- c. **System Logging:** The wallet system must maintain comprehensive operational logs covering all transactions, errors, system events, and API calls. Logs must be **searchable and accessible** to the operations and technical teams within **1 minute** of an event occurring.
- d. **Deployment Downtime:** Wallet system updates and patches must be deployable with a maximum of **15 minutes planned downtime** per release. Zero-downtime deployment must be targeted for minor updates and patches.

7. Compliance:

- a. **RBI PPI Compliance:** The wallet system must fully comply with RBI's Prepaid Payment Instrument guidelines including transaction limits, KYC requirements, and reporting obligations at all times.
- b. **AML & CFT Compliance:** The wallet system must support AML and CFT compliance requirements including transaction monitoring, suspicious activity flagging, and audit trail maintenance as required by RBI and FIU-IND.
- c. **GST Compliance:** All wallet transactions must be processed in compliance with Indian GST regulations. GST treatment for each transaction type must be confirmed by the Finance and Legal team prior to go-live.
- d. **Data Retention Compliance:** Wallet transaction data must be retained for a minimum of **5 years** in line with RBI financial data retention guidelines. The system must support data lifecycle management to ensure retained data remains accessible and auditable throughout the retention period.
- e. **Audit Trail Compliance:** The wallet system must maintain a complete, immutable, and tamper-proof audit trail for all wallet events. Audit logs must be available for regulatory inspection within **24 hours** of a formal request.

8. Disaster Recovery:

- a. **Recovery Point Objective (RPO):** The wallet system must have a maximum **Recovery Point Objective (RPO) of 1 hour** — meaning in the event of a system failure, no more than 1 hour of wallet transaction data can be lost.
- b. **Recovery Time Objective (RTO):** The wallet system must have a maximum **Recovery Time Objective (RTO) of 4 hours** — meaning the system must be fully restored and operational within 4 hours of a critical failure.
- c. **Daily Data Backup:** Wallet transaction data must be backed up at least once every **24 hours**. Backup integrity must be verified after every backup cycle. Backup storage must comply with RBI data localisation requirements ensuring all backup data remains within India.

- d. **Disaster Recovery Testing** The disaster recovery plan must be **tested at least once every 6 months** to validate RTO and RPO targets. Test results must be documented and reviewed by the DevOps and business teams.
- e. **Fallback Payment Option** In the event of wallet system unavailability, the checkout page must automatically **display alternative payment methods** (credit card, COD) to ensure customers can still complete purchases without disruption.

Risks:

Category: Regulatory & Compliance

1. PPI License Approval Delayed

- a. **Description:** RBI PPI license approval is delayed beyond the expected timeline
- b. **Details:** The wallet top-up feature requires a valid PPI license from RBI before it can legally operate in India. RBI license approvals are subject to regulatory review timelines entirely outside the project team's control, making delays highly probable. Any delay will prevent the top-up feature from going live as planned, directly impacting business revenue targets, user experience, and competitive positioning.
- c. **Impact:** High
- d. **Risk Level:** Critical
- e. **Mitigation & Contingency:** Initiate PPI license application at the earliest possible stage, well before development begins. Engage a regulatory consultant with RBI experience to guide and expedite the application. Track license status at every steering committee meeting. If approval is delayed, launch wallet in Phase 1 with core payment and refund features only. Resume top-up go-live immediately upon license grant without requiring additional development effort.
- f. **Risk Owner:** Legal & Compliance Officer

2. RBI Regulatory Changes Post Go-Live

- a. **Description:** RBI introduces new guidelines or amendments to PPI regulations after wallet go-live
- b. **Details:** RBI periodically updates payment and PPI regulations in response to market developments and emerging risks. Any regulatory change post go-live may require system modifications, process changes, or compliance updates within mandated timelines. Non-compliance with updated regulations could result in financial penalties, suspension of wallet operations, or reputational damage to the business.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Assign Legal & Compliance team to actively monitor RBI regulatory updates and circulars. Build wallet system with configurable parameters to accommodate regulatory changes without major rework. If a regulatory change is announced, immediately assess impact and initiate an emergency change request to implement required changes within mandated timelines.

- f. **Risk Owner:** Legal & Compliance Officer

3. Non-Compliance with RBI Minimum KYC Thresholds

- a. **Description:** Wallet transactions exceed RBI minimum KYC thresholds before full KYC is implemented
- b. **Details:** Since full KYC is not available at launch, the wallet will operate within RBI minimum KYC transaction limits. If wallet adoption is higher than anticipated, users may approach or exceed these thresholds faster than expected, creating a direct regulatory compliance violation. This risk is moderately probable given that transaction volumes are difficult to predict accurately at launch.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Implement hard system controls to enforce RBI minimum KYC transaction limits from day one. Monitor wallet transaction volumes against thresholds in real time with automated alerts when users approach 80% of their limit. If thresholds are breached, temporarily suspend wallet top-up for affected users and notify them to complete full KYC verification to restore full wallet functionality.
- f. **Risk Owner:** Legal & Compliance Officer

4. Vendor Non-Compliance with RBI Guidelines

- a. **Description:** Selected wallet provider or payment gateway found to be non-compliant with RBI guidelines after onboarding
- b. **Details:** If the wallet provider or payment gateway does not hold valid RBI compliance certifications, the business is exposed to regulatory risk by association. While this risk has a low probability since compliance is verified during vendor selection, the consequences of discovering non-compliance after onboarding are severe — including potential suspension of wallet operations.
- c. **Impact:** High
- d. **Risk Level:** Medium
- e. **Mitigation & Contingency:** Include RBI compliance verification as a mandatory criterion in the vendor selection process. Obtain and validate compliance certificates from all vendors prior to contract signing. If non-compliance is discovered post-onboarding, immediately suspend vendor engagement and initiate an emergency vendor replacement process to minimize operational disruption.
- f. **Risk Owner:** Legal & Compliance Officer / Project Sponsor

Category: Vendor & Third Party Risks

1. Wallet Provider Selection Delayed

- a. **Description:** Wallet technology provider selection takes longer than planned delaying development start
- b. **Details:** Vendor selection is currently in progress and is the highest priority external dependency for this project. Delays in shortlisting, evaluating, and

onboarding the wallet provider will directly push back the development start date and subsequently the go-live date. This risk is highly probable given that vendor evaluation processes typically involve multiple rounds of assessment, commercial negotiations, and legal contract reviews.

- c. **Impact:** High
- d. **Risk Level:** Critical
- e. **Mitigation & Contingency:** Define and communicate a clear vendor selection timeline with a firm deadline. Initiate vendor evaluation process immediately with a shortlist of pre-qualified providers. Fast-track contract negotiations by involving legal and procurement teams early. If vendor selection is delayed beyond the defined deadline, escalate to Project Sponsor for immediate intervention and reassess project timeline accordingly.
- f. **Risk Owner:** Project Sponsor / Project Manager

2. Wallet Provider API Integration Complexity

- a. **Description:** Wallet provider API integration is more complex than anticipated causing development delays
- b. **Details:** Once the wallet provider is selected, the complexity of their API integration may exceed initial estimates. Incomplete documentation, poorly designed APIs, or unexpected technical constraints from the provider side could significantly increase development effort and timeline. This risk is moderately probable as API complexity is difficult to assess accurately before vendor selection is finalized.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Include API documentation quality and sandbox environment availability as mandatory evaluation criteria during vendor selection. Conduct a technical spike or proof of concept with the selected vendor's API before committing to the full development timeline. If integration complexity exceeds estimates, reassess development timeline and resource requirements and escalate to Project Sponsor for timeline or budget adjustment.
- f. **Risk Owner:** Solution Architect / Development Team Lead

3. Wallet Provider System Unavailability

- a. **Description:** Wallet provider system experiences downtime or instability impacting wallet operations in production
- b. **Details:** The wallet feature's availability is directly dependent on the wallet provider's system uptime. Any unplanned downtime or instability on the provider's side will result in wallet transactions failing, impacting user experience and potentially causing revenue loss. This risk is moderately probable as even well-established providers experience occasional service disruptions.
- c. **Impact:** High
- d. **Risk Level:** High

- e. **Mitigation & Contingency:** Define and enforce strict SLA requirements for system availability during vendor contract negotiations. Implement real-time monitoring of wallet provider API health. If provider downtime occurs, automatically display alternative payment methods (credit card, COD) at checkout and notify users of temporary wallet unavailability with an estimated resolution time.
- f. **Risk Owner:** DevOps Team / Vendor Account Manager

4. Third Party Sandbox Instability

- a. **Description:** Wallet provider or payment gateway sandbox environment is unstable during testing phase
- b. **Details:** Integration testing and QA activities are entirely dependent on third-party sandbox environments being stable and fully functional. Sandbox instability — including downtime, data resets, or incomplete test scenario coverage — will directly delay testing progress and impact go-live readiness. This risk is moderately probable as sandbox environments are typically lower priority for vendors than production systems.
- c. **Impact:** Medium
- d. **Risk Level:** Medium
- e. **Mitigation & Contingency:** Include sandbox availability and stability as a contractual requirement with the wallet provider. Establish a dedicated vendor support contact for sandbox issues. If sandbox instability impacts testing progress, escalate to vendor account manager immediately and assess timeline impact. Maintain a buffer in the testing schedule to accommodate sandbox-related delays.
- f. **Risk Owner:** DQA Team Lead / Vendor Account Manager

Category: Technical Risks

1. Wallet Integration Complexity Higher Than Estimated

- a. **Description:** Overall wallet integration complexity exceeds initial estimates causing development delays and budget overrun
- b. **Details:** The wallet implementation involves multiple system integrations including the wallet provider, payment gateway, OMS, notification system, and reporting infrastructure. The combined complexity of these integrations may exceed initial estimates, particularly given that the wallet provider is not yet selected. This risk is moderately probable and could result in timeline delays and potential budget overrun on a fixed budget project.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Conduct a detailed technical design and estimation exercise once the wallet provider is selected and API documentation is available. Include integration complexity as a key assessment criterion during vendor selection. If complexity exceeds estimates, reassess scope and consider

deferring lower priority features to Phase 2 to protect the core delivery timeline and budget.

- f. **Risk Owner:** Solution Architect / Development Team Lead

2. Concurrent Transaction Race Conditions

- a. **Description:** Concurrent wallet transactions cause race conditions resulting in incorrect balance updates
- b. **Details:** When multiple transactions hit the wallet simultaneously — for example a payment and a refund processing at the same time — race conditions can occur that result in incorrect balance calculations or duplicate debits. This is a well-known technical challenge in payment systems and is moderately probable without proper concurrency controls in place from the start.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Define concurrency handling requirements clearly in the functional specifications. Ensure the development team implements appropriate locking mechanisms and transaction isolation controls. Include concurrent transaction scenarios in QA and performance testing. If race conditions are identified during testing, prioritize resolution before go-live as this is a critical financial data integrity issue.
- f. **Risk Owner:** Solution Architect / Development Team Lead

3. Payment Gateway Transaction Failure Rate

- a. **Description:** Payment gateway transaction failure rate exceeds acceptable thresholds during wallet top-up
- b. **Details:** Wallet top-up relies on the payment gateway processing credit card transactions successfully. If the gateway experiences higher than expected failure rates due to technical issues, network problems, or bank declines, users will be unable to top-up their wallets, directly impacting wallet adoption and user experience. This risk is low probability under normal conditions but increases during peak traffic periods.
- c. **Impact:** Medium
- d. **Risk Level:** Medium
- e. **Mitigation & Contingency:** Monitor payment gateway transaction success rates in real time with automated alerts for failure rate thresholds. Implement a user-friendly retry mechanism for failed top-up transactions. If failure rates exceed defined thresholds, escalate to gateway provider immediately and assess whether a backup gateway is required.
- f. **Risk Owner:** DevOps Team / Payment Gateway Account Manager

4. Performance Degradation Under Peak Load

- a. **Description:** Wallet system performance degrades under peak load conditions such as sales events

- b. **Details:** During high-traffic events such as sales promotions, the wallet system may experience significantly higher transaction volumes than normal operating conditions. If the system is not adequately load tested and optimized, performance degradation could result in slow response times, transaction failures, and poor user experience at the most critical business moments. This risk is moderately probable for any payment system that has not undergone rigorous performance testing.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Conduct comprehensive load and performance testing simulating peak traffic scenarios before go-live. Validate that the system meets all NFR performance benchmarks under peak load. If performance issues are identified during testing, optimize system performance before go-live. If degradation occurs in production during a peak event, activate infrastructure scaling measures immediately and communicate expected resolution time to business stakeholders.
- f. **Risk Owner:** DevOps Team / Solution Architect

Category: Business & Scope Risks

1. Scope Creep

- a. **Description:** Uncontrolled addition of new requirements post BRD sign-off leads to scope creep
- b. **Details:** Wallet implementations tend to attract additional feature requests from stakeholders once development begins — such as loyalty integration, cashback features, or additional payment methods. On a fixed budget project, scope creep is particularly dangerous as it can consume budget and timeline without formal approval. This risk is highly probable given the number of stakeholders involved and the open business decisions still pending.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Enforce a formal change control process from BRD sign-off onwards. All new requirements must go through the Change Control Board for assessment of budget, timeline, and resource impact before approval. If scope creep occurs despite controls, prioritize in-scope features for delivery and defer new additions to Phase 2 to protect the fixed budget and go-live date.
- f. **Risk Owner:** Project Manager / Business Analyst

2. Delayed Business Decisions on Open Items

- a. **Description:** Unresolved business decisions on open items delay requirements finalization and development start.
- b. **Details:** Several key business decisions remain open at this stage — including top-up payment methods, COD refund eligibility, and KYC implementation approach. Delays in resolving these decisions will prevent requirements from being finalized, directly delaying development start and potentially causing

rework if decisions change after development has begun. This risk is moderately probable given the number of open items and stakeholder availability constraints.

- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Formally document all open business decisions with assigned owners and resolution deadlines. Escalate unresolved items to the Project Sponsor at each steering committee meeting. If decisions remain unresolved beyond their deadline, make a conservative default assumption to unblock development and flag the assumption for stakeholder review at the earliest opportunity.
- f. **Risk Owner:** Project Manager / Business Analyst

3. Low Wallet Adoption by Users

- a. **Description:** Wallet feature sees low user adoption post launch failing to meet business objectives.
- b. **Details:** Despite technical delivery, the wallet may see lower than expected adoption if users are not aware of it, do not trust it, or do not find it convenient enough compared to existing payment methods. Low adoption directly impacts the return on investment of the wallet implementation and may raise questions about the business case. This risk is moderately probable for any new payment feature without a supporting launch and marketing strategy.
- c. **Impact:** Medium
- d. **Risk Level:** Medium
- e. **Mitigation & Contingency:** Ensure the wallet feature is prominently positioned at checkout with clear messaging on its benefits. Coordinate with the marketing team to plan a wallet launch campaign including user education and incentives. If adoption is lower than expected post launch, conduct user research to identify barriers and implement targeted improvements in a subsequent release.
- f. **Risk Owner:** Product Owner / Marketing Team

Category: Data Risks

1. Wallet Balance Discrepancies

- a. **Description:** Wallet balance discrepancies arise between internal records and payment gateway records
- b. **Details:** Reconciliation discrepancies between internal wallet transaction records and payment gateway records can occur due to timing differences, failed callbacks, or data mapping errors. Unresolved discrepancies create financial exposure and erode trust in the wallet system. This risk is moderately probable in any payment system that relies on real-time data exchange between multiple systems.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Implement automated daily reconciliation between internal wallet records and gateway records. Define clear reconciliation rules and

exception handling processes with the Finance team prior to go-live. If discrepancies are identified, freeze affected transactions pending investigation and resolve within the agreed SLA. All discrepancies must be logged and reported to the Finance team for review.

- f. **Risk Owner:** Finance Team / Solution Architect

2. Data Loss During System Failure

- a. **Description:** Wallet transaction data is lost or corrupted during a system failure or outage
- b. **Details:** In the event of an unexpected system failure during transaction processing, wallet transaction data may be lost or corrupted if adequate backup and recovery mechanisms are not in place. For a financial system, even a small amount of data loss is unacceptable and could result in incorrect wallet balances, reconciliation failures, and regulatory non-compliance. This risk has a low probability but a very high impact.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Implement daily data backups with integrity verification as defined in the NFRs. Ensure RPO of 1 hour and RTO of 4 hours are met through regular disaster recovery testing. If data loss occurs, immediately activate the disaster recovery plan, notify affected users, and initiate a manual reconciliation exercise to restore accurate wallet balances.
- f. **Risk Owner:** DevOps Team / Solution Architect

3. Unauthorized Access to Wallet Transaction Data

- a. **Description:** Unauthorized access to wallet transaction data results in data breach or financial fraud
- b. **Details:** Wallet transaction data contains sensitive financial information that is a high-value target for malicious actors. Unauthorized access — whether through external attacks or internal misuse — could result in financial fraud, data breach, regulatory penalties, and significant reputational damage. This risk is low probability given the security controls in scope but the impact is severe enough to warrant active monitoring.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Implement all security NFRs including encryption, role-based access control, session management, and API security from day one. Conduct a penetration test prior to go-live. Monitor access logs for unusual activity in real time. If unauthorized access is detected, immediately revoke access, freeze affected wallet accounts, notify the Legal & Compliance team, and initiate a security incident response process in line with regulatory reporting obligations.
- f. **Risk Owner:** IT Security Team / Legal & Compliance Officer

Category: Project & Delivery Risks

1. Fixed Budget Overrun

- a. **Description:** Project costs exceed the approved fixed budget due to scope changes or integration complexity
- b. **Details:** As a fixed budget project, any unplanned scope additions, integration complexities, or rework resulting from late requirement changes could exhaust the budget before all features are delivered. This risk is moderately probable given the number of open business decisions, the pending vendor selection, and the complexity of a wallet implementation with regulatory obligations.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Enforce strict change control from BRD sign-off. Maintain a budget contingency reserve of at least 10-15% of total project budget for unforeseen costs. Track budget utilization at every steering committee meeting. If budget overrun is anticipated, immediately escalate to Project Sponsor and assess scope reduction options to deliver the highest priority features within the remaining budget.
- f. **Risk Owner:** Project Sponsor / Project Manager

2. UAT Delays

- a. **Description:** Business stakeholder UAT participation is delayed or extended beyond the agreed window
- b. **Details:** UAT requires active participation from business stakeholders who have competing priorities alongside project responsibilities. Delayed UAT start, slow feedback cycles, or extended defect resolution periods can push the go-live date significantly. This risk is highly probable based on common patterns in enterprise project delivery.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Secure formal commitment from business stakeholders for UAT participation before the UAT phase begins. Define a clear UAT schedule with agreed entry and exit criteria. Build a buffer period into the project timeline to accommodate UAT extensions. If UAT is delayed beyond the agreed window, escalate to the Project Sponsor and assess go-live date impact.
- f. **Risk Owner:** Project Manager / Product Owner

3. Go-Live Readiness Gaps

- a. **Description:** Critical go-live readiness gaps are identified too late to resolve before the planned go-live date
- b. **Details:** Go-live readiness encompasses technical deployment, business process readiness, customer support training, monitoring setup, and regulatory compliance sign-off. If any of these readiness elements are incomplete at the planned go-live date, launching prematurely could result in operational failures,

poor user experience, and compliance exposure. This risk is moderately probable given the number of parallel workstreams involved.

- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Develop a comprehensive go-live readiness checklist covering all technical, operational, and compliance requirements. Conduct a formal go-live readiness review at least 2 weeks before the planned date to identify and address gaps. If critical gaps are identified too close to go-live, defer the launch date rather than risk a poor or non-compliant go-live.
- f. **Risk Owner:** Project Manager / Product Owner

4. Post Go-Live Critical Defects

- a. **Description:** Critical defects in wallet functionality are identified after go-live impacting users and business operations.
- b. **Details:** Despite thorough testing, critical defects may surface in production due to real-world usage patterns, edge cases, or integration behaviors that were not covered in testing. For a financial system, critical defects — such as incorrect balance updates, failed transactions, or reconciliation errors — can have immediate financial and reputational consequences. This risk is low probability but always present in any production deployment.
- c. **Impact:** High
- d. **Risk Level:** High
- e. **Mitigation & Contingency:** Implement a comprehensive testing strategy covering functional, integration, performance, and security testing prior to go-live. Establish a dedicated hypercare team for the first 4 weeks post go-live to monitor and respond rapidly to production issues. Define a clear severity classification and response SLA for post go-live defects. If a critical defect is identified, assess whether a hotfix can be deployed rapidly or whether the affected wallet feature needs to be temporarily suspended pending resolution.
- f. **Risk Owner:** Project Manager / Development Team Lead

Trade-offs Summary:

1. Phased Launch vs Full Feature Launch

- a. **Context:** The wallet implementation involves multiple features — core wallet payment, refund, and top-up — each with varying levels of complexity and dependency. The top-up feature is additionally constrained by the pending PPI license from RBI. The business wants to go live as soon as possible, creating a tension between launching a complete wallet solution and launching quickly with core features only.
- b. **Options:**
 - i. **Phased Launch:** Launch core wallet payment and refund features first without top-up. Top-up goes live in Phase 2 once the PPI license is granted.
 - 1. **Advantage:** Faster go-live, lower initial risk, no regulatory blocker.

2. **Disadvantage:** Wallet utility is limited without top-up — users can only spend refund credits, reducing initial adoption.
- ii. **Feature Launch:** Wait until all features including top-up are ready and PPI license is granted before launching.
 1. **Advantage:** Complete wallet experience from day one, stronger user adoption potential.
 2. **Disadvantage:** Significantly delayed go-live due to PPI license uncertainty, higher risk of timeline and budget overrun.
- c. **Recommended Decision:** Option A — Phased Launch. Given the business priority of speed to market and the uncertainty around PPI license timelines, a phased launch is the most pragmatic approach. It gets the wallet in front of users quickly while managing regulatory risk responsibly.
- d. **Implication:** The business should accept that initial wallet utility will be limited to refund credits only until top-up is available. A clear communication plan should be in place to manage user expectations at launch.
- e. **Decision Owner:** Project Sponsor / Product Owner

2. Speed to Market vs Comprehensive Testing Coverage

- a. **Context:** Given the business priority of launching as soon as possible, there is a natural tension between thorough testing coverage and a faster go-live date. A wallet system handling real financial transactions demands rigorous testing, but extensive testing cycles consume significant time.
- b. **Options:**
 - i. **Accelerated Testing:** Conduct focused testing covering critical payment flows, balance management, and security scenarios. Defer edge case and regression testing to a post-launch stabilization phase.
 1. **Advantage:** Faster go-live, core functionality validated.
 2. **Disadvantage:** Higher risk of post-launch defects in edge case scenarios, potential user-facing issues, and financial exposure from untested scenarios.
 - ii. **Comprehensive Testing:** Conduct full end-to-end testing covering all functional, integration, performance, security, and edge case scenarios before go-live.
 1. **Advantage:** Higher confidence in system quality, lower post-launch defect risk.
 2. **Disadvantage:** Longer testing cycle, delayed go-live, higher testing resource cost.

- iii. **Recommended Decision:** Option B — Comprehensive Testing with a risk-based approach. Given that this is a financial system handling real money, compromising on testing coverage is not advisable. However a risk-based testing approach — prioritizing the highest risk and highest impact scenarios first — can optimize testing efficiency without sacrificing quality. Critical payment flows, balance management, security, and reconciliation must be fully tested before go-live.
- iv. **Implication:** The business should accept a realistic go-live timeline that accommodates thorough testing. Rushing testing to meet an arbitrary deadline introduces financial and reputational risk that far outweighs the benefit of an earlier launch.
- v. **Decision Owner:** Project Sponsor / QA Team Lead

3. Full KYC at Launch vs Minimum KYC with Transaction Limits

- a. **Context:** Full KYC verification is not currently available on the platform and building it from scratch would require significant additional scope, effort, and timeline. RBI permits wallet operations under minimum KYC thresholds as a starting point. This creates a trade-off between launching with full KYC compliance and launching faster with minimum KYC controls.
- b. **Options:**
 - i. **Minimum KYC at Launch:** Launch wallet with RBI minimum KYC transaction limits enforced — maximum wallet balance of ₹10,000 and cumulative annual transactions of ₹1,00,000. Build full KYC as a separate workstream.
 - 1. **Advantage:** Faster go-live, no KYC infrastructure needed at launch.
 - 2. **Disadvantage:** Transaction limits restrict wallet utility for higher-value users, potentially limiting adoption among premium customer segments.
 - ii. **Full KYC at Launch:** Build and integrate a full KYC verification process as part of this implementation before go-live.
 - 1. **Advantage:** No transaction limits, full wallet utility from day one, stronger regulatory compliance posture.
 - 2. **Disadvantage:** Significantly increases project scope, timeline, and budget. May not be feasible within the fixed budget.
 - iii. **Recommended Decision:** Option A — Minimum KYC at Launch. Given the fixed budget constraint and speed to market priority, building full KYC as part of this implementation is not practical. Minimum KYC thresholds are sufficient for the initial launch and provide a compliant operating framework while full KYC is developed as a separate workstream.

iv. **Implication:** The business should accept that high-value users will be restricted by RBI minimum KYC transaction limits until full KYC is implemented. A clear roadmap and timeline for full KYC should be communicated internally and to users approaching their transaction limits.

v. **Decision Owner:** Project Sponsor / Legal & Compliance Officer

4. **COD Refund to Wallet vs Card-Only Refund to Wallet**

a. **Context:** COD is one of the two existing payment methods on the platform. Whether COD orders are eligible for wallet refunds is an open business decision. Including COD refunds to wallet adds complexity — as it involves converting physical cash payments into digital wallet credits — but excluding it limits the refund experience for a significant portion of users.

b. Options:

i. **Card-Only Refund to Wallet:** Restrict wallet refunds to credit card orders only at launch. COD refunds continue to follow the existing refund process.

1. **Advantage:** Simpler implementation, no COD-to-digital conversion complexity, no accounting complications.
2. **Disadvantage:** COD users cannot benefit from wallet refunds, creating an inconsistent refund experience across payment methods.

ii. **COD Refund to Wallet:** Allow COD order refunds to be credited to the wallet.

1. **Advantage:** Consistent refund experience across all payment methods, higher wallet balance accumulation driving future wallet usage.
2. **Disadvantage:** Adds accounting and reconciliation complexity — converting physical cash collections into digital wallet credits requires additional financial controls and potentially regulatory considerations.

iii. **Recommended Decision:** Option A — Card-Only Refund to Wallet at Launch. COD-to-wallet refund involves financial and accounting complexity that is better addressed in a subsequent phase once the core wallet is stable. This keeps the initial implementation clean and manageable within the fixed budget.

iv. **Implication:** The business should accept that COD users will not benefit from wallet refunds at launch. This should be communicated clearly in the user-facing wallet feature documentation to avoid confusion and customer support escalations.

- v. **Decision Owner:** Project Sponsor / Finance Head

5. Advanced Fraud Detection vs Basic Rule-Based Fraud Controls

- a. **Context:** Wallet systems are high-value fraud targets. Advanced AI/ML based fraud detection offers superior protection but requires significant additional investment in data infrastructure and expertise. Basic rule-based fraud controls are simpler and faster to implement but offer less sophisticated protection.
- b. **Options:**
 - i. **Basic Rule-Based Fraud Controls:** Implement configurable rule-based fraud detection triggers as defined in scope.
 - 1. **Advantage:** Faster implementation, lower cost, sufficient for initial launch volumes.
 - 2. **Disadvantage:** Less adaptive than AI/ML systems, may miss sophisticated fraud patterns that evolve over time.
 - ii. **Advanced AI/ML Fraud Detection:** Build or integrate an AI/ML based real-time fraud scoring system.
 - 1. **Advantage:** Superior fraud detection, adaptive to evolving fraud patterns.
 - 2. **Disadvantage:** Significantly higher cost, longer implementation timeline, requires data science capability — not feasible within fixed budget.
- c. **Recommended Decision:** Option A — Basic Rule-Based Fraud Controls at Launch. Advanced fraud detection is not feasible within the fixed budget and launch timeline. Rule-based controls are sufficient for initial launch and provide a solid foundation. Advanced fraud detection should be planned as a Phase 2 initiative once the wallet has sufficient transaction data to train ML models effectively.
- d. **Implication:** The business should accept a higher fraud exposure risk compared to platforms with advanced fraud detection. The Risk & Fraud team should actively monitor wallet transactions manually during the initial launch period and refine fraud rules based on observed patterns.
- e. **Decision Owner:** Project Sponsor / Risk & Fraud Team

Success Metrics:

1. Adoption Metrics

a. Wallet Activation Rate

- i. **Description:** Measures the percentage of eligible registered users who activate their wallet within 90 days of go-live. This is the primary adoption indicator showing how well the wallet feature is being discovered and

adopted by the existing user base.

- ii. **Target:** 25% of eligible registered users activate wallet within 90 days of go-live.
- iii. **Measurement Period:** Weekly tracking, 90-day cumulative target
- iv. **Data Source:** Wallet transaction system — wallet account creation records
- v. **Owner:** Product Owner

b. **Wallet Feature Awareness Rate**

- i. **Description:** Measures the percentage of registered users who visit the wallet section or view their wallet balance at least once within 90 days of go-live. This indicates whether users are aware of the wallet feature even if they have not yet transacted.
- ii. **Target:** 40% of registered users interact with the wallet feature within 90 days of go-live
- iii. **Measurement Period:** Weekly tracking, 90-day cumulative target
- iv. **Data Source:** Platform analytics / user behaviour tracking
- v. **Owner:** Product Owner

c. **Wallet Repeat Usage Rate**

- i. **Description:** Measures the percentage of users who use the wallet for more than one transaction within 90 days of their first wallet payment. Repeat usage indicates that users find value in the wallet and are choosing it over other payment methods.
- ii. **Target:** 50% of users who make their first wallet payment make at least one additional wallet payment within 90 days
- iii. **Measurement Period:** Monthly tracking, 90-day cumulative target
- iv. **Data Source:** Wallet transaction system
- v. **Owner:** Product Owner

2. **Transaction Metrics**

a. **Wallet Payment Adoption at Checkout**

- i. **Description:** Measures the percentage of total orders placed using wallet as the payment method within 90 days of go-live. This directly reflects how well the wallet is competing with existing payment methods at the point of purchase.
- ii. **Target:** Wallet accounts for a minimum of 15% of total orders within 90 days of go-live
- iii. **Measurement Period:** Weekly tracking, 90-day cumulative target
- iv. **Data Source:** Order Management System / Wallet transaction system

- v. **Owner:** Product Owner / Finance Head

b. **Wallet Transaction Success Rate**

- i. **Description:** Measures the percentage of wallet payment attempts that complete successfully without failure or timeout. A high success rate is critical for user trust and continued wallet adoption.
- ii. **Target:** Minimum 99.5% wallet transaction success rate within 90 days of go-live
- iii. **Measurement Period:** Daily monitoring, weekly reporting
- iv. **Data Source:** Wallet transaction system
- v. **Owner:** DevOps Team / Product Owner

c. **Average Wallet Transaction Value**

- i. **Description:** Measures the average order value of transactions paid via wallet. This helps the business understand the spending behavior of wallet users and whether the wallet is being used for high-value or low-value purchases.
- ii. **Target:** Establish baseline within first 30 days and track for 10% growth in average wallet transaction value by day 90
- iii. **Measurement Period:** Monthly tracking
- iv. **Data Source:** Wallet transaction system / Order Management System
- v. **Owner:** Finance Head / Product Owner

3. **Financial Metrics**

a. **Checkout Conversion Rate Improvement**

- i. **Description:** Measures the improvement in overall checkout conversion rate attributable to the introduction of the wallet payment option. Wallet payments reduce checkout friction by eliminating the need for card details entry on repeat purchases, which directly improves conversion. This is the primary business objective metric for this implementation.
- ii. **Target:** 5-8% improvement in overall checkout conversion rate within 90 days of go-live compared to pre-launch baseline
- iii. **Measurement Period:** Weekly tracking, 90-day comparison against pre-launch baseline
- iv. **Data Source:** Platform analytics / Order Management System
- v. **Owner:** Product Owner / Finance Head

b. **Wallet-Driven Revenue Retention**

- i. **Description:** Measures the percentage of refund credits issued to wallets that are subsequently spent on the platform within 90 days. This indicates how effectively the wallet retains revenue that would otherwise leave the

platform via traditional refund methods.

- ii. **Target:** 70% of wallet refund credits are spent on the platform within 90 days of being issued
- iii. **Measurement Period:** Monthly tracking
- iv. **Data Source:** Wallet transaction system / Order Management System
- v. **Owner:** Finance Head / Product Owner

4. Operational Metrics

a. Wallet System Uptime

- i. **Description:** Measures the actual uptime of the wallet system in production against the 99.9% NFR target. This is the primary operational health indicator for the wallet post go-live.
- ii. **Target:** Minimum 99.9% uptime within 90 days of go-live
- iii. **Measurement Period:** Daily monitoring, monthly reporting
- iv. **Data Source:** System monitoring and alerting infrastructure
- v. **Owner:** DevOps Team

b. Reconciliation Accuracy Rate

- i. **Description:** Measures the percentage of wallet transactions that are successfully reconciled between internal wallet records and payment gateway records without manual intervention. A high reconciliation accuracy rate indicates healthy data integrity and reduces Finance team workload.
- ii. **Target:** 99.95% automated reconciliation accuracy within 90 days of go-live with all exceptions resolved within 24 hours
- iii. **Measurement Period:** Daily monitoring, weekly reporting
- iv. **Data Source:** Wallet transaction system / Payment gateway records
- v. **Owner:** Finance Team

c. Failed Transaction Recovery Rate

- i. **Description:** Measures the percentage of failed wallet transactions that are successfully recovered and completed through the automated retry mechanism without user intervention. A high recovery rate reduces user frustration and minimizes revenue loss from failed payments.
- ii. **Target:** Minimum 80% of retryable failed transactions recovered automatically within 90 days of go-live
- iii. **Measurement Period:** Weekly tracking
- iv. **Data Source:** Wallet transaction system
- v. **Owner:** DevOps Team / Product Owner

5. Customer Experience Metrics

a. Wallet-Related Customer Support Ticket Volume

- i. **Description:** Measures the volume of customer support tickets raised specifically related to wallet issues — including failed transactions, balance discrepancies, refund delays, and access issues. A low and declining ticket volume indicates a stable and user-friendly wallet experience.
- ii. **Target:** Wallet-related support tickets should represent no more than 5% of total support ticket volume within 90 days of go-live
- iii. **Measurement Period:** Weekly tracking
- iv. **Data Source:** Customer Support ticketing system
- v. **Owner:** Customer Support Team / Product Owner

b. Wallet User Satisfaction Score

- i. **Description:** Measures user satisfaction with the wallet feature through post-transaction surveys or NPS (Net Promoter Score) data. This provides direct qualitative feedback on the wallet experience from the user's perspective.
- ii. **Target:** Minimum CSAT score of 4.0 out of 5.0 or NPS of +30 for wallet feature within 90 days of go-live
- iii. **Measurement Period:** Monthly tracking
- iv. **Data Source:** Post-transaction user surveys / NPS platform
- v. **Owner:** Product Owner / Customer Support Team

6. Compliance Metrics

a. KYC Threshold Breach Rate

- i. **Description:** Measures the number of users who approach or breach RBI minimum KYC transaction thresholds within 90 days of go-live. A rising breach rate signals that full KYC implementation needs to be accelerated to maintain regulatory compliance.
- ii. **Target:** Zero KYC threshold breaches within 90 days of go-live. Automated alerts triggered when users reach 80% of their threshold
- iii. **Measurement Period:** Daily monitoring, weekly reporting
- iv. **Data Source:** Wallet transaction system
- v. **Owner:** Legal & Compliance Officer

b. Fraud Detection Trigger Rate

- i. **Description:** Measures the number of fraud detection triggers fired within 90 days of go-live — including the breakdown of triggers by type, severity, and outcome. This metric helps the Risk & Fraud team assess the

effectiveness of fraud rules and identify patterns requiring rule refinement.

- ii. **Target:** Establish baseline fraud trigger rate within first 30 days. False positive rate should remain below 2% to avoid impacting legitimate user transactions
- iii. **Measurement Period:** Daily monitoring, weekly reporting
- iv. **Data Source:** Wallet transaction system / Fraud detection rules engine
- v. **Owner:** Risk & Fraud Team

All success metrics are indicative and will be validated against actual baseline data during discovery and post-deployment monitoring.